

AD CLICK FRAUD DETECTION

Professional Machine Learning Project Roadmap

A controlled, leakage-safe and business-focused plan for building a portfolio-ready fraud analytics system.

Prepared for Nandan M

Version 1.0 | September 2026

How to use this roadmap

This document is the governing plan for the project. Each stage combines learning, implementation, validation, documentation and a Git milestone. A stage is complete only after its completion gate is satisfied; unresolved irregularities are documented and resolved before progression.

Project positioning

Recommended title: **Ad Click Fraud Detection: A Leakage-Safe, Cost-Sensitive Machine Learning Pipeline**

Portfolio description

An end-to-end machine-learning system that identifies suspicious advertising clicks using behavioural and time-based features, time-aware validation, cost-sensitive threshold selection and explainable predictions.

Scope classification

The project belongs primarily to **fraud analytics and trust & safety**, with meaningful cybersecurity relevance. It addresses bots, click farms, repeated suspicious traffic, spoofed identifiers and attribution abuse, but should not be presented as a purely cybersecurity project.

Core principles

- Business definition before modelling.
- Prediction-time realism: features may use only information available when a score is produced.
- Time-based validation instead of a default random split.
- Metrics suited to severe class imbalance; accuracy is never the headline metric.
- Reproducible code, configuration and documented decisions.
- Conventional machine learning first, with an architecture open to future AI and production extensions.

End-to-end controlled pipeline

Phase	Purpose	Primary output
1. Frame	Define user, decision, target and costs	Project charter
2. Acquire	Select, verify and document data	Raw data contract
3. Understand	Audit quality, imbalance and time coverage	Data audit + EDA
4. Engineer	Create leakage-safe behavioural features	Feature dataset
5. Validate	Simulate future scoring with temporal splits	Train/validation/test sets
6. Model	Establish baselines and compare candidates	Experiment results
7. Decide	Select threshold using business costs	Operating policy
8. Explain	Interpret predictions and analyse errors	Model report
9. Productize	Package pipeline, API/app and tests	Working application
10. Monitor	Track drift, performance and retraining	Monitoring design
11. Present	Create portfolio and interview narrative	GitHub release

Controlled stages

The implementation is divided into 17 stages. Stages 1-8 establish trustworthy data and evaluation; stages 9-12 build and select the model; stages 13-16 turn it into a reusable portfolio product; stage 17 defines future enhancements.

Estimated effort

A careful portfolio implementation is typically a multi-week project. Effort depends heavily on dataset size and available hardware. We will prefer correctness and evidence over rushing to a complex model.

Recommended technology stack

Area	Recommended technology	Role
Language	Python	Core implementation
Exploration	Jupyter Notebook	Audits, EDA and controlled experiments
Data	Pandas, NumPy; optional Polars	Transformation and memory-efficient processing
Visualisation	Matplotlib, Seaborn, Plotly	Static analysis and interactive charts
ML	scikit-learn	Preprocessing, baselines, metrics and pipelines
Boosting	LightGBM or XGBoost	High-performance tabular modelling
Explainability	SHAP	Global and local explanations
Tracking	MLflow	Parameters, metrics and model artifacts
Application	Streamlit	Fraud-monitoring portfolio interface
Optional API	FastAPI	Reusable prediction service
Configuration	YAML	Paths, features, parameters and thresholds
Testing	pytest	Data, feature and inference tests
Quality	Ruff / Black	Consistent maintainable Python
Versioning	Git and GitHub	History, collaboration and presentation
Environment	venv or Conda	Reproducible dependencies

Tooling rule

Tools are introduced only when they add demonstrable value. MLflow, SHAP, FastAPI and monitoring are not decorations; each must support a documented project need.

Target repository architecture

Path	Purpose
README.md	Project story, results, setup and usage
configs/	Data, feature, model and threshold configuration
data/raw/	Immutable source data; large files excluded from Git
data/interim/	Validated intermediate datasets
data/processed/	Model-ready features and split manifests
notebooks/	Numbered exploration and experiment notebooks
src/data/	Ingestion, schema and validation logic
src/features/	Reusable, leakage-safe feature transformations
src/models/	Training, prediction and threshold logic
src/evaluation/	Metrics, plots, cost and error analysis
src/monitoring/	Drift and performance checks
app/	Streamlit application
tests/	Unit and integration tests
models/	Serialized model and preprocessing artifacts
reports/figures/	Publication-ready charts
docs/project_decisions.md	Decision log and rationale
docs/project_progress.md	Stage status, blockers and next actions

Notebook-to-source rule

Notebooks support exploration. Any logic required for repeatable training or inference is moved into `src/`, configured externally and tested.

Stage 1: Project definition and business problem

Outcome: a precise prediction decision, stakeholder and measurable project objective.

Concepts to learn

- Binary classification
- Decision thresholds
- Fraud operations
- Cost-sensitive learning

Work to complete

- Identify the intended stakeholder: ad platform, advertiser or fraud analyst.
- Define what action follows a high-risk prediction: allow, review, throttle or block.
- Specify the scoring moment and which fields exist at that moment.
- Define false-positive and false-negative consequences.
- Write an explicit scope, assumptions, exclusions and success criteria.

Required deliverables

- docs/project_charter.md
- Initial business-cost matrix
- Draft model card context section

Completion gate

- Target meaning is unambiguous.
- No post-outcome information is allowed at scoring time.
- Business and ML success criteria are measurable.

Suggested Git milestone: docs: define fraud detection problem and success criteria

Stage 2: Dataset selection and validation

Outcome: a defensible dataset whose target and limitations are understood.

Work to complete

- Compare candidate datasets by target quality, licensing, size, realism and reproducibility.
- For TalkingData, distinguish confirmed fraud from non-attribution or non-conversion.
- Record source, version, license, download procedure and checksums.
- Assess local memory/storage constraints and choose full-data or sampled-development strategy.
- Create an initial column-level data dictionary.

Required deliverables

- Dataset comparison table
- Dataset card and limitation statement
- Raw-file manifest and checksum
- Data dictionary

Completion gate

- Dataset use is permitted and reproducible.
- Target is not overstated as perfect fraud ground truth.
- Compute plan is feasible on available hardware.

Suggested Git milestone: docs: document dataset choice and limitations

Stage 3: Repository and environment setup

Outcome: a clean, reproducible development foundation.

Work to complete

- Create the agreed folder structure.
- Initialize Git and a Python environment.
- Add dependency specification, .gitignore and configuration templates.
- Prevent raw data, secrets, caches and large model artifacts from accidental commits.
- Add a minimal command or script proving imports and paths work.

Required deliverables

- Repository skeleton
- requirements or environment file
- Configuration files
- Starter README
- Environment verification

Completion gate

- A fresh environment can be created from documented instructions.
- No raw dataset or secrets are tracked.
- Paths are configuration-driven, not machine-specific.

Suggested Git milestone: chore: initialize reproducible project structure

Stage 4: Data ingestion and integrity checks

Outcome: raw data loads reliably without silent corruption.

Work to complete

- Load data with explicit dtypes and memory controls.
- Validate schema, row count, file size and essential columns.
- Profile missing values, duplicates, malformed timestamps and category cardinality.
- Preserve raw data as immutable.
- Create automated validation checks and a data-quality report.

Required deliverables

- Reusable ingestion module
- Schema contract
- Data-quality report
- Validated interim dataset or manifest

Completion gate

- Counts reconcile with source files.
- Timestamps parse consistently.
- Unexpected schema changes fail loudly.
- Raw data remains unchanged.

Suggested Git milestone: feat: add validated data ingestion pipeline

Stage 5: Exploratory data analysis

Outcome: evidence-based understanding of class behaviour, traffic and time patterns.

Work to complete

- Quantify class imbalance and target prevalence.
- Analyse click volume and target rate by time, app, channel, device, OS and IP.
- Investigate repeated-click velocity and suspicious bursts.
- Inspect temporal coverage, distribution shifts and rare categories.
- Separate descriptive observations from causal claims.

Required deliverables

- 02_eda.ipynb
- Reusable EDA figures
- Written findings and open questions
- Irregularity log

Completion gate

- Every chart answers a stated question.
- Axes, denominators and time ranges are correct.
- Potential leakage and dataset artifacts are flagged before modelling.

Suggested Git milestone: analysis: complete click fraud exploratory analysis

Stage 6: Leakage-safe feature engineering

Outcome: behavioural features reproducible at the declared scoring moment.

Work to complete

- Build calendar features such as hour and weekday.
- Create historical click counts, unique counts, frequency encodings and interaction features.
- Calculate time since prior click and rolling-window velocity using past data only.
- Fit aggregate encodings on training data and apply them to later periods.
- Define unseen-category behaviour and feature naming conventions.

Required deliverables

- Feature specification
- Reusable feature transformer(s)
- Feature lineage table
- Leakage tests

Completion gate

- Every feature has an availability timestamp and formula.
- No future events or target-derived statistics leak into inputs.
- Training and inference generate identical feature schemas.

Suggested Git milestone: feat: build leakage-safe behavioural features

Stage 7: Time-based split design

Outcome: validation that realistically estimates future performance.

Work to complete

- Order observations chronologically.
- Choose train, validation and final test windows.
- Consider a temporal gap when adjacent events could share information.
- Freeze the final test set until model and threshold decisions are complete.
- Record class rates and category coverage by split.

Required deliverables

- Split configuration
- Split manifest
- Distribution comparison table
- Rationale for dates and gaps

Completion gate

- No timestamp overlap exists.
- Test data has not influenced features, tuning or threshold selection.
- Each split is large enough for reliable minority-class evaluation.

Suggested Git milestone: feat: implement temporal train validation test split

Stage 8: Preprocessing pipeline

Outcome: one deterministic transformation path for training and inference.

Work to complete

- Handle missing and unknown categorical values explicitly.
- Select suitable encoding for each model family.
- Apply scaling only where the algorithm benefits.
- Manage high-cardinality fields without exploding memory.
- Serialize preprocessing together with model inputs.

Required deliverables

- Preprocessing pipeline
- Feature schema artifact
- Unit tests for missing and unseen values
- Memory/runtime benchmark

Completion gate

- No preprocessing is fitted on validation or test data.
- Unknown values do not break inference.
- Output column order and dtypes are stable.

Suggested Git milestone: feat: add reproducible preprocessing pipeline

Stage 9: Baseline modelling

Outcome: simple reference points that establish whether complexity adds value.

Work to complete

- Create a naive prevalence baseline.
- Create a transparent rule-based fraud-risk baseline.
- Train class-weighted logistic regression.
- Evaluate all baselines on the same temporal validation window.
- Record runtime, memory, metrics and limitations.

Required deliverables

- Baseline models
- Comparison table
- Confusion matrices
- Experiment records

Completion gate

- Baselines are reproducible.
- Accuracy is not used as the main success measure.
- Later models must beat meaningful baselines, not only the naive classifier.

Suggested Git milestone: model: establish naive rule and logistic baselines

Stage 10: Advanced model experiments

Outcome: a justified champion candidate for tabular fraud scoring.

Work to complete

- Train a tree ensemble and a gradient-boosting candidate such as LightGBM or XGBoost.
- Use class weighting and controlled sampling strategies.
- Tune a focused set of parameters using temporal validation.
- Track experiments with MLflow after the baseline workflow is stable.
- Compare discrimination, calibration, runtime, memory and maintainability.

Required deliverables

- Experiment matrix
- Tracked runs and artifacts
- Candidate comparison report
- Provisional champion model

Completion gate

- Identical splits and metrics are used across models.
- No test-set tuning has occurred.
- Selected complexity produces meaningful value over the baseline.

Suggested Git milestone: model: compare advanced temporal fraud classifiers

Stage 11: Evaluation and threshold selection

Outcome: an operating threshold tied to business priorities.

Work to complete

- Report precision, recall, F1/F-beta, PR-AUC and ROC-AUC.
- Plot precision-recall and threshold trade-offs.
- Evaluate top-k or review-capacity scenarios.
- Create a cost matrix and estimate cost or savings across thresholds.
- Assess probability calibration when risk scores are interpreted probabilistically.

Required deliverables

- Final validation scorecard
- Threshold analysis
- Cost curve
- Documented operating threshold

Completion gate

- Threshold is chosen on validation data only.
- Metric definitions and positive-class meaning are explicit.
- The recommended threshold reflects a stated operational objective.

Suggested Git milestone: eval: select operating threshold using fraud costs

Stage 12: Explainability and error analysis

Outcome: understandable model behaviour and known failure modes.

Work to complete

- Use permutation importance or SHAP for global drivers.
- Explain representative individual predictions.
- Inspect false positives and false negatives by time and category.
- Evaluate performance across apps, channels, devices and periods.
- Check whether suspicious proxies or unstable identifiers dominate.

Required deliverables

- Feature-importance report
- SHAP visuals
- Segment scorecard
- Error taxonomy and mitigation ideas

Completion gate

- Explanations match transformed features.
- Claims are phrased as associations, not proof of causality.
- Material subgroup or temporal weaknesses are documented.

Suggested Git milestone: analysis: explain model and document failure modes

Stage 13: Reusable training and inference pipeline

Outcome: repeatable model creation and consistent scoring outside notebooks.

Work to complete

- Create command-line training and evaluation entry points.
- Bundle preprocessing, model, feature list, threshold and metadata.
- Add deterministic seeds and configuration validation.
- Log data version, code version and model version.
- Implement single-record and batch prediction interfaces.

Required deliverables

- Training command
- Versioned model bundle
- Batch prediction script
- Inference contract
- Reproducibility report

Completion gate

- A clean run reproduces results within expected tolerance.
- Inference does not depend on notebook state.
- Input validation produces clear errors.

Suggested Git milestone: feat: package reproducible training and inference

Stage 14: Streamlit fraud-monitoring application

Outcome: a portfolio interface that demonstrates decisions, not only predictions.

Work to complete

- Create an executive overview with traffic and risk KPIs.
- Add batch upload and scoring workflow.
- Show risk score, predicted class and main contributing factors.
- Provide threshold and cost simulation.
- Add model-performance and data-drift views with careful labels.

Required deliverables

- Streamlit application
- Screenshots or demo GIF
- Sample input file
- App usage documentation

Completion gate

- Predictions match the pipeline output.
- UI prevents invalid schemas and communicates limitations.
- Charts remain readable across filters and screen sizes.

Suggested Git milestone: feat: add interactive fraud monitoring app

Stage 15: Testing and quality assurance

Outcome: evidence that core logic remains correct as the project evolves.

Work to complete

- Test schema validation, temporal splits, rolling features and leakage boundaries.
- Test preprocessing on missing and unseen categories.
- Test prediction shape, probability range and threshold logic.
- Add an end-to-end smoke test on a small fixture.
- Run formatting and lint checks.

Required deliverables

- pytest suite
- Small synthetic fixtures
- Quality commands
- Test coverage summary

Completion gate

- Critical transformations have tests.
- All tests pass from a clean environment.
- No secrets, local paths or raw data appear in Git history.

Suggested Git milestone: test: validate data features and inference pipeline

Stage 16: Documentation and portfolio presentation

Outcome: a recruiter can understand, reproduce and evaluate the work quickly.

Work to complete

- Write README sections for problem, architecture, dataset, methods, results, limitations and usage.
- Include architecture and pipeline diagrams.
- Document decisions, experiments and unresolved limitations.
- Create an interview narrative: problem, obstacles, choices, results and next steps.
- Tag a stable release after final verification.

Required deliverables

- Complete README
- Model card
- Data card
- Project decisions and progress logs
- Interview presentation notes
- Release checklist

Completion gate

- Claims match measured results.
- Setup steps work.
- Links and images render correctly.
- No private or oversized artifacts are committed.

Suggested Git milestone: docs: finalize portfolio story and reproducibility guide

Stage 17: Monitoring and future AI extensions

Outcome: a credible evolution plan without overstating the current implementation.

Work to complete

- Specify input drift, prediction drift and delayed-label performance monitoring.
- Define alert thresholds and retraining triggers.
- Plan analyst feedback and rule-plus-model decisions.
- Consider streaming features, graph fraud networks and sequence models.
- Optionally add an LLM-generated analyst explanation layer grounded strictly in model evidence.

Required deliverables

- Monitoring design
- Retraining policy
- Future architecture plan
- Extension backlog

Completion gate

- Current capabilities and future proposals are clearly separated.
- AI explanations cannot override risk policy or invent evidence.
- Monitoring accounts for delayed or imperfect labels.

Suggested Git milestone: docs: define monitoring and future extension strategy

Evaluation framework

Because fraud datasets are highly imbalanced, a model can achieve very high accuracy while detecting almost no suspicious activity. The positive class must be defined explicitly before any metric is interpreted.

Metric	Use	Caution
Precision	Share of flagged clicks that are truly positive	Higher precision can reduce unnecessary blocks or reviews
Recall	Share of positive clicks successfully detected	Higher recall may increase false alerts
F1 / F-beta	Balances precision and recall	Beta should match business priority
PR-AUC	Primary ranking metric under strong imbalance	Always compare with prevalence baseline
ROC-AUC	Overall ranking discrimination	Can look optimistic under severe imbalance
Calibration	Quality of predicted probability	Discrimination and calibration are different
Cost / savings	Connects errors to business impact	Assumptions must be stated and sensitivity-tested
Recall at review capacity	Operational performance at limited queue size	Capacity must reflect a realistic scenario

Minimum reporting set

PR-AUC, precision, recall, threshold, confusion matrix, class prevalence, temporal test period and a cost/capacity interpretation.

Leakage prevention checklist

- Write down the exact timestamp at which a prediction is made.
- Exclude conversion/attribution fields unavailable at that timestamp.
- Compute counts and rates using only earlier observations.
- Fit encoders, imputers and scalers on training data only.
- When using target encoding, generate training values out-of-fold and use only historical labels.
- Do not tune features, hyperparameters or threshold on the final test set.
- Keep duplicate or linked events from silently crossing boundaries where relevant.
- Test feature functions with synthetic timestamps designed to expose future-data access.
- Record each feature's source columns, time window, fitting scope and missing-value behaviour.

The key question

For every feature, ask: **Could the production system know this value at the moment it must score the click?** If the answer is no, the feature is leakage regardless of its predictive power.

Governance and continuity

The project should remain understandable even when work continues in a new chat or after a long break.

Control file	Update rule
docs/project_progress.md	Update after each session: completed work, active issue and next action
docs/project_decisions.md	Record decision, alternatives, rationale and consequences
docs/irregularities.md	Log anomaly, evidence, resolution and affected outputs
docs/experiment_log.md	Record data version, split, features, model, parameters and results
README.md	Update at meaningful milestones; keep public claims aligned with final evidence

New-chat handoff template

Continue my Ad Click Fraud Detection portfolio project. Read README.md, docs/project_progress.md and docs/project_decisions.md first. The last completed stage is [stage]. The current blocker or next task is [task]. Preserve all recorded decisions and do not advance past unresolved irregularities.

Session close routine

- Run the relevant validation and tests.
- Update progress, decision and irregularity logs.
- Review Git diff for accidental data or secrets.
- Commit one coherent milestone with a descriptive message.
- Record the exact next action.

Milestone map

Milestone	Stages	Portfolio evidence
M1 - Foundation	1-3	Charter, dataset decision, reproducible repository
M2 - Trusted data	4-5	Schema validation, audit and fraud-focused EDA
M3 - Feature system	6-8	Leakage-safe features and temporal validation
M4 - Model evidence	9-12	Baselines, champion, threshold, explanations
M5 - Product	13-15	Reusable pipeline, application and tests
M6 - Portfolio release	16	README, model/data cards and interview narrative
M7 - Evolution	17	Monitoring and future-AI plan

Definition of portfolio-ready

- The target and dataset limitations are stated honestly.
- Temporal leakage controls are demonstrated, not merely claimed.
- The model beats meaningful baselines on a locked future test period.
- Threshold choice connects to cost or operational capacity.
- Errors and weaknesses are analysed.
- Training and inference are reproducible outside notebooks.
- The application uses the same verified pipeline.
- Tests pass and documentation matches actual results.

Immediate next action

Begin Stage 2 by comparing candidate datasets. Confirm target definition, licensing, file size, columns, time coverage and hardware feasibility before downloading or designing features.